

Chapter 2: Supervised Learning

Universidad de Salamanca

Faculty of Sciences

Advanced Programming

Diego M. Jiménez-Bravo · dmjimenez@usal.es

Outline

- 1 Introduction
- 2 2.1 Regression
- 3 2.2 Classification

Outline

- 1 Introduction
- 2 2.1 Regression
- 3 2.2 Classification

What is supervised learning?

In supervised learning we have:

- **Data**: input features, denoted X .
- **Labels**: the correct answers associated with each data point, denoted y .
- A **goal**: learn the relationship $X \rightarrow y$ from already-solved examples.

It is like learning with a “supervisor” who tells you, during training, whether your answer to each example is correct or not.

There are two broad types of problems, depending on the nature of the label y we want to predict:

Regression (continuous value)

- House price
- Tomorrow's temperature

Classification (category)

- Is this email spam?
- Is this image a cat, a dog, or a bird?

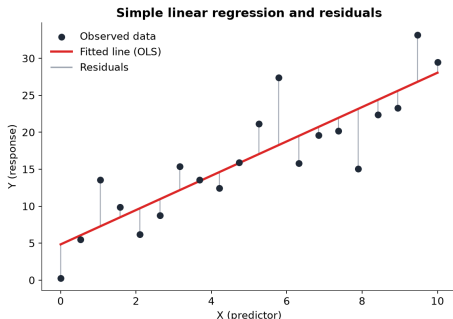
Outline

- 1 Introduction
- 2 2.1 Regression**
- 3 2.2 Classification

Linear regression

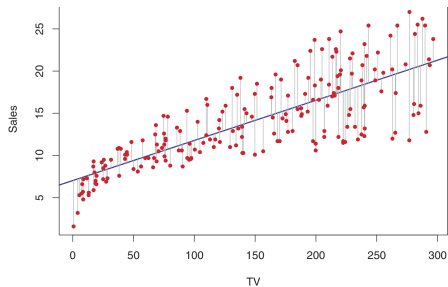
Regression is the problem of predicting a **continuous value** Y from one or more predictor variables X . Linear regression assumes an approximately linear relationship between them.

- **Simple**: a single predictor, $Y \approx \beta_0 + \beta_1 X$.
- **Multiple**: one coefficient per available feature, $Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \epsilon$.
- β_0 (**intercept**) and $\beta_1, \dots, \beta_p$ (**weights**) are what the model learns from the data. In a p -dimensional space, the model geometrically represents a **hyperplane**.



Real example: the *Advertising* dataset

A classic example from the literature is the *Advertising* dataset, where a line is fit to predict sales (Sales) from TV advertising spending (TV). Each vertical gray line is a **residual**: the distance between the observed value and the one predicted by the line.



Source: James, Witten, Hastie & Tibshirani (2013), *An Introduction to Statistical Learning*, Fig. 3.1. Freely distributed textbook (statlearning.com).

Ordinary Least Squares (OLS)

Ordinary Least Squares (OLS) is the standard approach for training linear models.

- A **residual** $e_i = y_i - \hat{y}_i$ is the difference between the real and predicted value.
- OLS finds the parameters that make those residuals, squared and summed, as small as possible (this sum is called the RSS).
- Unlike other models, linear regression has an **exact solution**: a formula that computes the best parameters directly in a single step, with no trial and error needed.

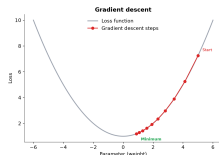
Example: with just a few houses (m² and price), OLS finds in a single computation the line that best summarizes that relationship.

Gradient descent

For massive datasets or models with thousands of parameters, solving the exact OLS equation is too costly. **Gradient descent** is the iterative optimization algorithm that powers much of modern machine learning.

- The algorithm starts from random weights and nudges them little by little, taking small steps in the direction that most reduces the error.
- The **learning rate** sets the size of each step: too high and the algorithm can overshoot and fail to converge; too low and training will be painfully slow.
- Variants: *Batch* (uses all the data at every step), *Stochastic* (SGD, one sample per step), and *Mini-batch* (a small group, the most common choice in practice).

Example: it's like walking down a foggy mountain: at each step you feel the slope under your feet and move toward where it descends most, until you reach the valley.



Polynomial regression

When the data shows curvature, linear models can be extended through **feature engineering**: powers of the original predictors (X^2 , X^3 , ...) are added as new input columns.

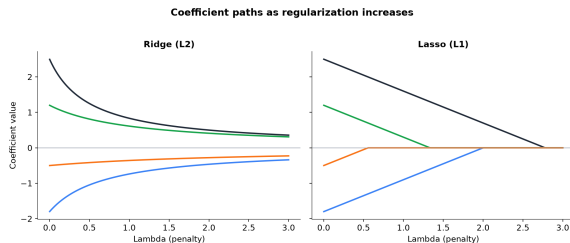
Although the resulting model is non-linear with respect to the original variables, it remains **linear in its parameters**, so it can still be trained with OLS or gradient descent without any changes.

The risk: increasing the polynomial degree increases the model's flexibility, which can lead it to memorize the noise in the training data (overfitting) instead of capturing the real trend.

Regularization: *Ridge* and *Lasso*

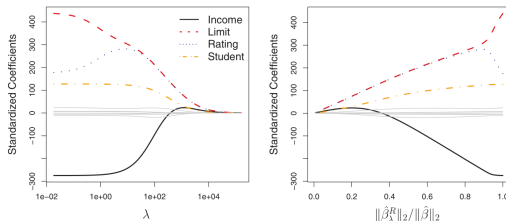
Regularization techniques add a penalty to the cost function for having large weights, forcing the model to stay simpler and generalize better.

- **Ridge**: shrinks all coefficients toward zero, without ever eliminating them entirely; works very well when many predictors are related to each other.
- **Lasso**: penalizes in a slightly different way, and can drive some coefficients exactly to zero, performing **automatic variable selection**.
- **Elastic Net**: combines both penalties.



Ridge on real data: the *Credit* dataset

As the value of λ (the penalty strength) increases, the coefficients shrink progressively toward zero. This reduces the model's **variance** at the cost of a small increase in **bias** — exactly the trade-off we seek when regularizing.



Source: James, Witten, Hastie & Tibshirani (2013), *An Introduction to Statistical Learning*, Fig. 6.4.

Validation and diagnostics in regression

- **MSE/RMSE**: heavily penalize large errors; RMSE is expressed in the same units as Y , which makes it easier to interpret.
- **MAE** (mean absolute error): the “typical” error size, more robust to outliers than MSE.
- R^2 : the proportion of Y 's variation the model manages to explain. 1 = perfect fit; 0 = as good as always predicting the mean.
- **Residual analysis**: if plotting the model's errors reveals a clear pattern instead of random noise, that's a sign the linear model is falling short.

Outline

- 1 Introduction
- 2 2.1 Regression
- 3 2.2 Classification**

Classification: predicting categories

In classification, the goal is to predict a **category** (class) instead of a continuous value. Typical examples: email classification (spam vs. not spam), medical classification (sick vs. healthy), or species classification (cat vs. dog). Unlike linear regression, classification starts by estimating the **probability** that a sample belongs to a particular category.

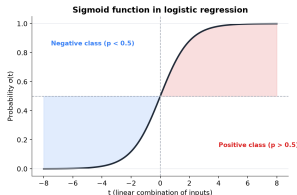
Logistic regression: the sigmoid function

Logistic regression uses the **sigmoid function** to turn a linear combination of the input features into a **probability**, always between 0 and 1:

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

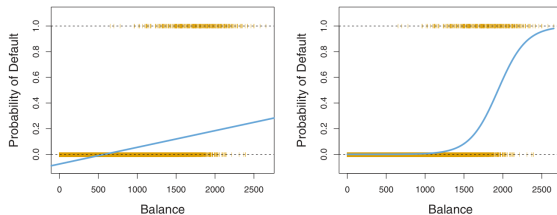
The larger t is, the closer the output gets to 1 (positive class); the smaller, the closer it gets to 0.

The model is trained, using gradient descent, by adjusting its weights so the predicted probabilities match the real labels (0 or 1) as closely as possible, heavily penalizing predictions that are confident but wrong.



Why not use linear regression?

Linear regression can predict probabilities below 0 or above 1, which makes no sense. Thanks to the sigmoid, logistic regression always keeps its output within the range $[0, 1]$. The example below compares both fits on the *Default* dataset (credit default): on the left, linear regression produces absurd probabilities; on the right, logistic regression always produces valid probabilities.

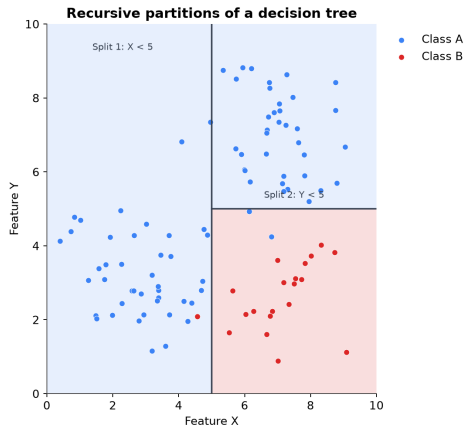


Source: James, Witten, Hastie & Tibshirani (2013), An Introduction to Statistical Learning, Fig. 4.2.

Decision trees

Trees recursively partition the input space into rectangular “boxes”, following a “divide and conquer” approach.

- **Construction:** the tree asks a series of binary questions (“is this variable above this value?”), picking at each step the one that best separates the classes.
- **Pruning:** to prevent overfitting, unnecessary branches are removed based on statistical tests or a validation set.

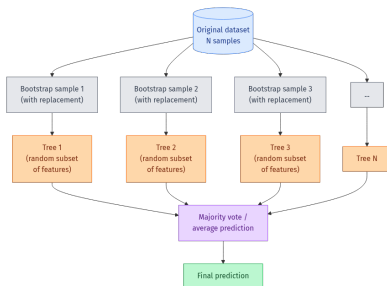


Example: “does the email contain the word ‘free’? → yes → does it have more than 3 links? → yes → spam.”

Random Forests

A random forest is a collection of many decision trees, designed to reduce variance and improve robustness compared to a single tree.

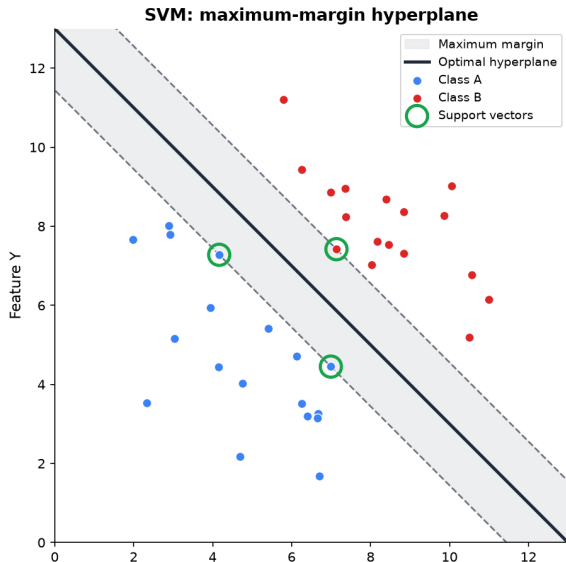
- **Bagging**: each tree is trained on a different *bootstrap* sample (obtained by sampling with replacement from the original set).
- **Decorrelation**: at each split, only a random subset of features is considered, to make sure the trees are diverse.
- Individual predictions are combined by **majority vote** (classification) or **averaging** (regression).



Support Vector Machines (SVM)

This model looks for the **hyperplane** that separates the classes with the **maximum margin** possible.

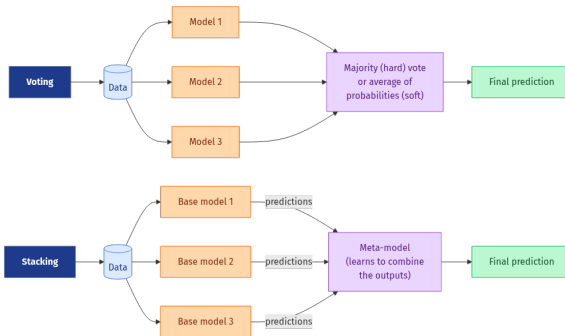
- The **support vectors** are the only samples that determine the boundary's position; moving any other point does not change the model.
- With data that isn't linearly separable, the **soft margin** allows some errors, and the **kernel trick** implicitly maps the data to a higher-dimensional space where it *is* separable.



Voting and stacking

Ensemble methods combine several models to obtain a prediction better than any of them individually.

- **Voting**: combines independent models with a fixed rule — majority (“hard”) vote or average of probabilities (“soft”).
- **Stacking**: trains a **meta-model** that learns, from data, how to combine the base models’ outputs — a learned rule, not a fixed one.



Boosting

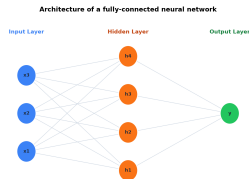
Unlike the parallel training of random forests, boosting trains models **sequentially**: each new model focuses on correcting the errors made by the previous one, giving more weight to misclassified instances. The final prediction combines all models through a weighted sum. Modern examples widely used in data competitions: **XGBoost** and **Gradient Boosting**.



Neural networks

Neural networks represent learning through successive layers of filtering.

- **Architecture:** an input layer, one or more hidden layers (densely connected), and an output layer.
- **Activation functions:** introduce non-linearity so complex patterns can be learned. **ReLU** is the standard for hidden layers; **Softmax** is used in the output layer for multi-class classification.
- **Backpropagation:** the central learning mechanism. It propagates the error backward from the output, layer by layer, adjusting the network's weights to reduce total loss.



Evaluation: validation protocols

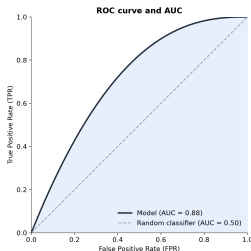
Just as in regression, the data must be split into independent partitions to measure generalization reliably:

- **Training, validation, and test:** one set for fitting weights, another for choosing hyperparameters, and a third, untouched, for the final evaluation.
- ***K*-fold cross-validation:** the recommended strategy when data is scarce.
- **Stratification:** in classification, each partition must preserve the original class proportions, especially when they are imbalanced.

Evaluation: success metrics

The same metrics from the previous chapter (accuracy, precision, recall, F1, ROC-AUC) apply here, and matter just as much:

- **Accuracy** can be misleading on imbalanced data: if 90% of samples belong to one class, always predicting that class gives 90% accuracy without having learned anything.
- That's why **precision** and **recall** are also worth checking, especially when the cost of a mistake depends on which class gets confused.



Evaluation doesn't happen only at the end: it should guide the entire development process.

- **Beat a baseline:** before moving to advanced models, check that you beat something very simple (e.g., always predicting the most common class).
- **Learning curves:** plotting training and validation error throughout training reveals the exact point where overfitting sets in.
- **Early stopping:** automatically halts training once the model stops improving on validation.
- **Error analysis:** manually reviewing the cases where the model fails helps understand what's confusing it.

Chapter summary

- In **regression** we predict a continuous value; in **classification**, a category.
- **OLS** gives an exact solution; **gradient descent** scales to massive data.
- **Regularization** (Ridge/Lasso) prevents overfitting by penalizing large weights.
- **Logistic regression** extends the idea of linear regression to probabilities via the sigmoid.
- **Trees**, **Random Forest**, and **SVM** are classic classification alternatives, each with its own strengths.
- **Ensemble** methods (voting, stacking, boosting) combine models to improve the result.
- **Neural networks** learn layered representations through backpropagation.
- Evaluating well — with the right splits and metrics — is just as important as training well.

Chapter 2: Supervised Learning

Universidad de Salamanca

Faculty of Sciences

Advanced Programming

Diego M. Jiménez-Bravo · dmjimenez@usal.es